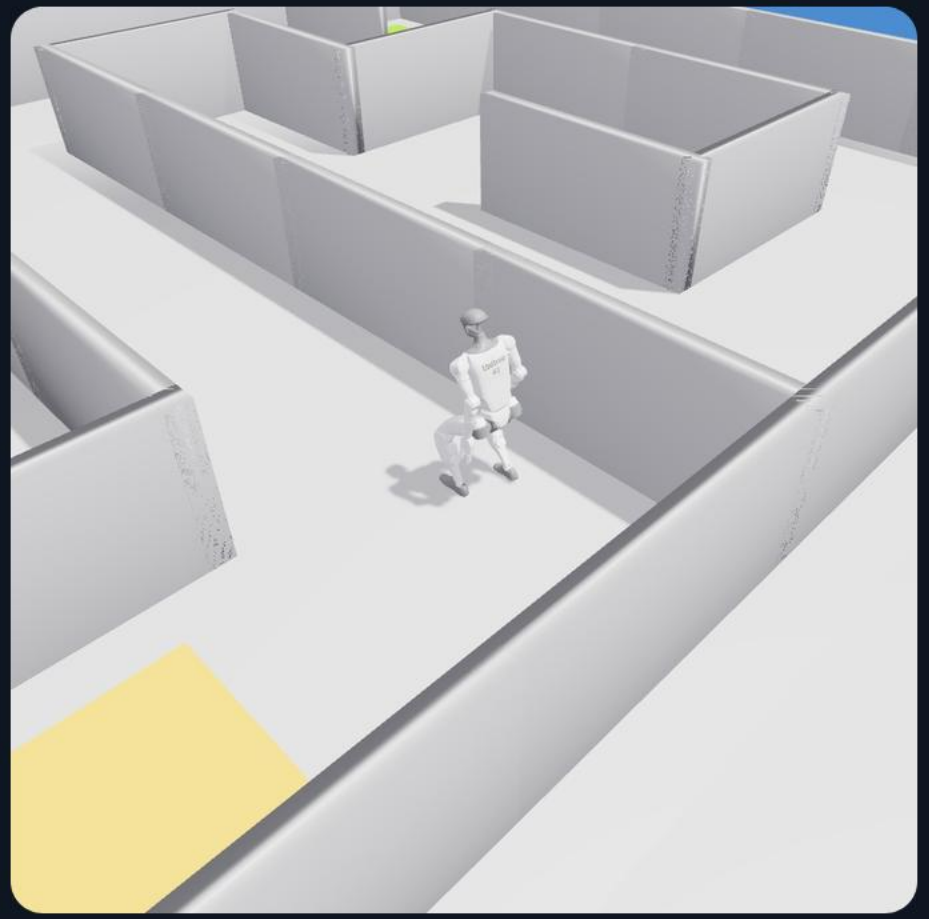
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

Simulation for Humanoid

Week 7 — Curriculum learning · G1 Maze Challenge

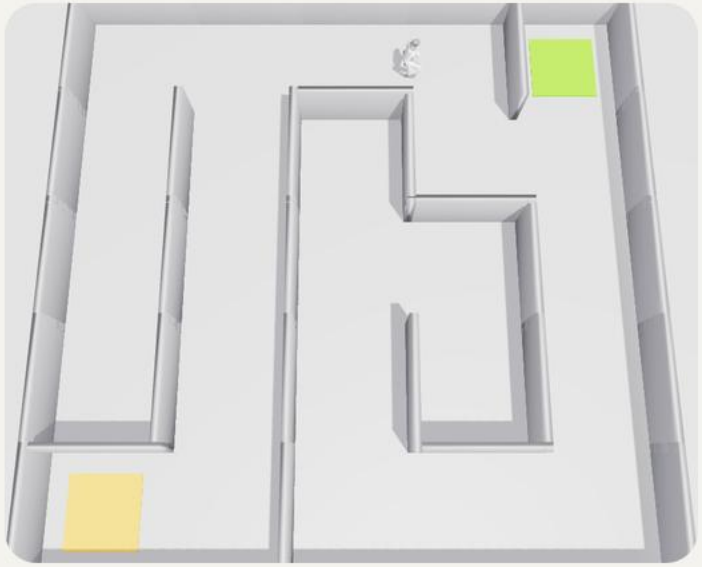
Isaac Lab 3.0 + Newton 2026 edition

Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai



WEEK 7

Agenda



01 Recap · เป้าหมายวันนี้

02 Curriculum learning คืออะไร

03 Curriculum ใน Isaac Lab 3

04 Lab 7.1 · G1 + curriculum 3 แบบ

05 G1 Maze Challenge · height scanner

06 Exercises

RECAP · WEEK 3 – 6

เรามี G1 ที่เดินตามคำสั่งได้แล้ว — แต่ยังต้องมีคนกดปุ่ม

Week 3 · play_interactive.py

ขับ G1 ด้วยคีย์บอร์ด

IJKL / UO → v_x, v_y, w_z → policy → 37 ข้อต่อ

Week 4 – 5

reward = สเปกของงาน

เปลี่ยน weight / เขียน reward ใหม่ แล้ววัดผลด้วย eval

Week 6

หุ่นของเราเอง

URDF ↔ USD · ยืดขา · เทรนใหม่บน Newton

วันนี้: วันนี้เราจะ (1) ทำให้การเทรน **ค่อย ๆ ยากขึ้นเอง** ระหว่างเทรน และ (2) เปลี่ยนคนกดปุ่มเป็น **autopilot** ที่ใช้ sensor หาทางออกจากเขาวงกต

THIS WEEK

Curriculum + Maze Challenge

ของเดิมใช้ Isaac Lab 2 + PhysX + Isaac Sim GUI → ฉบับนี้ port ทั้งหมดมา Isaac Lab 3 + Newton และทดสอบรันจริงทุกคำสั่ง

`CurriculumTermCfg`

7.0 · Concept + API

curriculum ทำงานตอนไหน · terrain_levels_vel · built-in ของ Isaac Lab 3

`G1-Curriculum-*`

7.1 · G1 + 3 terms

reward ramp · push แบบปรับตามการล้ม · ขยายช่วงความเร็ว

`G1-Maze-*`

7.2 · Maze Challenge

G1 เดินเองในเขาวงกต: height scanner → กำแพง → เลือกทาง → คำสั่งความเร็ว

LECTURE

Curriculum learning

เรียนจากง่ายไปยาก — แบบเดียวกับที่คนเรียนคณิตศาสตร์ ไม่ได้เริ่มจากแคลคูลัส

WHY CURRICULUM

ปล่อยให้หุ่นเจอโจทย์ยากตั้งแต่วันแรก → ล้มจนไม่ได้เรียน

Sparse reward

reward หายาก

ถ้าได้คะแนนก็ต่อเมื่อเดินถึงเป้า policy สุ่มแทบไม่เคยได้คะแนน → ไม่มีสัญญาณให้เรียน

Exploration

state space ใหญ่

37 ข้อต่อ × พื้นหลายแบบ × ความเร็วหลายค่า — สุ่มเดินไปเรื่อย ๆ ไม่เจอท่าที่ดี

Instability

ล้มตลอด

episode จบภายใน 1 วินาที ข้อมูลที่เก็บได้เป็นแค่ “ท่าล้ม” → เทรนนาน หรือไม่ converge

Curriculum เริ่มจากโจทย์ที่หุ่น **พอทำได้** แล้วเพิ่มความยาก **เมื่อทำได้ดีพอ** — ข้อมูลที่เก็บจึงมีทั้งความสำเร็จและความล้มเหลวพอดี ๆ

CURRICULUM IN ROBOTICS

เดินได้ก่อน → แล้วค่อยเดินบนพื้นยาก เร็ว และโดนผลัก

Stage 1

พื้นเรียบ · ช้า

flat terrain · v_x ≤ 0.5 m/s · ไม่มีแรงผลัก · reward ง่าย

Stage 2

เพิ่มความยาก

ทางลาด / ขั้นบันไดเตี้ย · v_x ≤ 1.0 · ผลักเบา ๆ

Stage 3

สมจริง

พื้นสุ่ม · v_x ≤ 1.5+ · ผลักแรง · penalty ครบชุด

สมดุล: ถ้าข้ามขั้น (โยนหุ่นเข้า Stage 3 ทันที) หุ่นแทบไม่มีโอกาสเรียน · ถ้าค้างที่ Stage 1 นานเกินไป หุ่นเก่งแค่พื้นเรียบ

WHAT CAN CHANGE

ปรับ “ความยาก” ได้จาก 4 ทาง

ปรับอะไร	ตัวอย่าง	curriculum term	manager ที่ถูกแก้
terrain (พื้น)	ระดับความยากของพื้น	terrain_levels_vel (มีให้แล้ว)	TerrainImporter
reward weight	น้ำหนักของ reward / penalty	modify_reward_weight · **RewardWeightRamp** (วันนี้)	RewardManager
disturbance	แรงผลัก, มวล, แรงเสียดทาน	**AdaptivePush** (วันนี้)	EventManager
command	ช่วงความเร็วที่สั่ง	**CommandRange** (วันนี้)	CommandManager
อื่น ๆ	ค่าใดก็ได้ใน config	modify_env_param / modify_term_cfg	ทุก manager

สังเกต: curriculum ไม่เปลี่ยน network หรือ PPO เลย — มันแก้ **config ของ environment ระหว่างเทรน** ผ่าน manager ต่าง ๆ

ISAAC LAB 3

CurriculumManager

curriculum เป็นแค่ฟังก์ชันที่ถูกเรียกตอนหุ่น reset — คืนค่าอะไรมา ค่านั้นไปขึ้นกราฟ TensorBoard

WHEN DOES IT RUN

ถูกเรียกใน _reset_idx — ก่อนล้างคะแนนของ episode

`step()`
physics × 4

→

`terminations`
ล้ม / หมดเวลา

→

`curriculum.compute(env_ids)`
เฉพาะตัวที่ reset

→

`events (reset)`
สุ่มท่าเริ่มใหม่

→

`rewards.reset`
ล้าง episode sums

`env_ids`

รู้ว่าใครเพิ่งจบ

ส่งมาแค่หุ่นที่เพิ่งจบ episode — นับได้ว่าล้มกี่ตัว เดินไกลเท่าไร

`episode sums`

ยังอ่านคะแนนได้

reward ของ episode ที่เพิ่งจบยังอยู่ใน reward_manager._episode_sums

`Curriculum/<name>`

ค่าที่ return → กราฟ

rsl_rl log ให้อัตโนมัติ · ดูใน TensorBoard ได้ทันที

นาฬิกา: env.common_step_counter = จำนวน step ตั้งแต่เริ่ม process (1 iteration = 24 step) — เริ่มนับ 0 ใหม่ทุกครั้งที่รัน แม้จะ fine-tune ต่อจาก checkpoint

CURRICULUMTERMCFG

ลงทะเบียน curriculum = เพิ่ม field ใน CurriculumCfg

```python
from isaaclab.managers import CurriculumTermCfg as CurrTerm

@configclass
class CurriculumCfg:
    # ชื่อ field = ชื่อ term = ชื่อกราฟ Curriculum/terrain_levels
    terrain_levels = CurrTerm(func=mdp.terrain_levels_vel)

# เพิ่ม term ทีหลังใน __post_init__ ก็ได้
self.curriculum.push_strength = CurrTerm(
    func=mdp.AdaptivePushCurriculum,     # ฟังก์ชัน หรือ class
    params={"max_vel": 1.5, "step": 0.1},  # ส่งเป็น keyword args
)
```

func

ฟังก์ชัน

f(env, env_ids, **params) — ไม่มีสถานะ (stateless)

ManagerTermBase

class

มี __init__ (เก็บสถานะ) + __call__ — Isaac Lab 3 ใช้แบบนี้กับ built-in ส่วนใหญ่

BUILT-IN · CORE/VELOCITY/MDP/CURRICULUMS.PY

terrain_levels_vel: เดินได้ไกล → ขยับไปพื้นที่ยากขึ้น

```
# compute the distance the robot walked
distance = torch.linalg.norm(asset.data.root_pos_w.torch[env_ids,
:2] - env.scene.env_origins[env_ids, :2], dim=1)
# robots that walked far enough progress to harder terrains
move_up = distance > terrain.cfg.terrain_generator.size[0] / 2
# robots that walked less than half of their required distance go
to simpler terrains
move_down = distance < torch.linalg.norm(command[env_ids, :2],
dim=1) * env.max_episode_length_s * 0.5
move_down *= ~move_up
# update terrain levels
terrain.update_env_origins(env_ids, move_up, move_down)
```

`move_up`

ไกลกว่าครึ่งแผ่น

ระยะ > size/2 (G1: 8 m → 4 m) → แถวที่ยากขึ้น

`move_down`

ไม่ถึงครึ่งของที่สั่ง

ระยะ < |v_cmd| × 20 s × 0.5 → แถวที่ง่ายกว่า

จำไว้: แถว (row) ของ terrain generator = ความยาก: difficulty = (row + U(0,1)) / num_rows เมื่อ terrain_generator.curriculum = True — ข้อจำกัด: term นี้ปรับได้แค่พื้น ไม่แตะ reward, แรงผลัก หรือความเร็ว

ISAAC LAB 3 · ISAACLAB.ENVS.MDP.CURRICULUMS

มีอะไรให้ใช้แล้วบ้าง

term	แบบ	ทำอะไร
modify_reward_weight	class	เปลี่ยน weight ครั้งเดียวเมื่อ step > num_steps
modify_env_param	class	แก้ค่าใดก็ได้ผ่าน address เช่น event_manager.cfg.x.params
modify_term_cfg	class	แบบเดียวกัน แต่เขียนสั้น: "rewards.feet_air_time.weight"
DifficultyScheduler	class	เก็บระดับความยากต่อหุ่น เลื่อนขึ้น/ลงตาม reward term ชื่อ success
terrain_levels_vel	function	ระดับพื้นตามระยะที่เดินได้ (locomotion)

Isaac Lab 2 (สไลด์เดิม)

เขียนเป็นฟังก์ชัน

modify_reward_weight(env, env_ids, term_name, weight, num_steps)

Isaac Lab 3

เป็น class

ชื่อเดิม พารามิเตอร์เดิม แต่ __init__ หา term cfg ไว้ก่อน · ใช้ใน CurrTerm ได้เหมือนเดิม

ที่มา: ~/IsaacLab/source/isaaclab/isaaclab/envs/mdp/curriculums.py · docs: isaac-sim.github.io/IsaacLab → How-to → Curriculum utilities

LAB 7.1

G1 + curriculum 3 แบบ

เขียน term เอง 3 ตัว แล้ว fine-tune G1 Rough ต่อจาก checkpoint ของ Isaac Lab — รันละ ~5 นาทีบนโน้ตบุ๊ก

```
g1_maze_lab/
├── pyproject.toml          # entry point isaaclab.tasks
├── pretrained/g1_rough/    # checkpoint G1 Rough (Isaac Lab, Newton)
├── scripts/
│   ├── eval_g1.py          # วัดความเร็ว / ล้ม / แรงผลัก
│   └── maze_autopilot.py   # G1 เดินเองในเขาวงกต
└── src/g1_maze_lab/
    ├── autopilot.py        # กำแพง → ทาง → คำสั่ง
    ├── terrains/maze.py     # sub-terrain เขาวงกต
    └── tasks/
        ├── curriculum/
        │   ├── mdp/curriculums.py
        │   ├── env_cfg.py
        │   └── agents/rsl_rl_ppo_cfg.py
        └── maze/  env_cfg.py · mdp/terminations.py
```

WEEK07 · G1_MAZE_LAB

project เดียว สองงาน

Lab 7.1 · curriculum

3 terms + 5 tasks G1-Curriculum-*

Lab 7.2 · maze

terrain + sensor + autopilot · 5 tasks G1-Maze-*

```
$ cd Week07_Curriculum_Maze/g1_maze_lab
$ uv sync   # ครั้งแรก
$ uv run isaaclab list_envs | grep G1-
```

TERM 1 · TASKS/CURRICULUM/MDP/CURRICULUMS.PY

RewardWeightRamp: ค่อย ๆ เพิ่ม weight แทนการกระโดด

```
class RewardWeightRampCurriculum(ManagerTermBase):
    def __init__(self, cfg, env):
        super().__init__(cfg, env)
        self._term_cfg =
env.reward_manager.get_term_cfg(cfg.params["term_name"])

    def __call__(self, env, env_ids, term_name, start_weight, end_weight,
                 start_step, end_step) → float:
        frac = (env.common_step_counter - start_step) / max(end_step -
start_step, 1)
        frac = min(max(frac, 0.0), 1.0)
        self._term_cfg.weight = start_weight + frac * (end_weight -
start_weight)
        env.reward_manager.set_term_cfg(term_name, self._term_cfg)
        return self._term_cfg.weight
```

`__init__`

หา term ครั้งเดียว

get_term_cfg("feet_air_time") เก็บไว้ใน self

`frac 0 → 1`

เส้นตรงตามเวลา

ก่อน start_step = ค่าเริ่ม · หลัง end_step = ค่าสุดท้าย · ใช้กับ G1: feet_air_time 0.25 → 1.0 ช่วง it. 50 → 150

TERM 2 · ADAPTIVE

AdaptivePush: ล้มน้อย → ผลักแรงขึ้น · ล้มเยอะ → ผลักเบาลง

```
fell = env.termination_manager.get_term(fall_term)[env_ids]
self._ended += len(env_ids)
self._fell += int(fell.sum())
if env.common_step_counter - self._last_update ⩾ update_every and
self._ended > 0:
    fall_rate = self._fell / self._ended
    if fall_rate < raise_below:
        self.level = min(self.level + step, max_vel)
    elif fall_rate > lower_above:
        self.level = max(self.level - step, 0.0)
    self._apply(event_name, env)
    self._ended, self._fell, self._last_update = 0, 0,
env.common_step_counter
return self.level
```

termination_manager

นับว่าใครล้ม

get_term("base_contact")[env_ids] = หุ่นที่เพิ่งจบเพราะลำตัวแตะพื้น

update_every = 480

ตัดสินใจทุก 20 iterations

< 5 % ล้ม → +0.1 m/s · > 20 % → −0.1 m/s · สูงสุด 1.5 m/s · _apply() เขียน velocity_range ของ event push_robot

TERM 3 · PERFORMANCE-BASED

CommandRange: ตามความเร็วได้ดี → ขยายช่วงความเร็วที่สั่ง

```
# episode sum / episode length = average reward per second; its
maximum is the weight itself
sums = env.reward_manager._episode_sums[reward_term][env_ids]
self._sum += float(sums.sum()) / env.max_episode_length_s
self._count += len(env_ids)
if env.common_step_counter - self._last_update ⩾ update_every:
    weight = env.reward_manager.get_term_cfg(reward_term).weight
    if self._sum / self._count > threshold * weight:
        lo, hi = self._ranges.lin_vel_x
        self._ranges.lin_vel_x = (lo, min(hi + step, final_max))
    self._sum, self._count, self._last_update = 0.0, 0,
env.common_step_counter
return self._ranges.lin_vel_x[1]
```

`episode sum ÷ 20 s`

คะแนนเฉลี่ยต่อวินาที

ค่าเต็ม = weight (เดินตรงคำสั่งเป๊ะทั้ง episode) → เทียบกับ 80 %

`update_every = 480`

1.0 → สูงสุด 2.0 m/s

+0.1 ไม่บ่อยกว่าทุก 20 it. (ฉบับแรกไม่มีบรรทัดนี้ — ดูหน้า Bug)

`command cfg`

แก้ ranges ตรง ๆ

get_term(...).cfg.ranges — ใช้ตอนสุ่มคำสั่งครั้ง

TASKS/CURRICULUM/ENV_CFG.PY

5 configs: Base + Reward / Push / Velocity / Full

```python
class G1CurriculumBaseEnvCfg(G1RoughEnvCfg):

    def __post_init__(self):
        super().__post_init__()
        self.scene.num_envs = 1024
        self.scene.terrain.terrain_generator.num_rows = 6
        self.scene.terrain.terrain_generator.num_cols = 10
        self.scene.terrain.max_init_terrain_level = 3

class G1CurriculumFullEnvCfg(G1CurriculumBaseEnvCfg):
    def __post_init__(self):
        super().__post_init__()
        self.curriculum.feet_air_time_ramp = reward_ramp_term()
        self.curriculum.push_strength = push_term()
        self.curriculum.command_vel_x = velocity_term()
```

G1RoughEnvCfg

เริ่มจาก G1 Rough

มี terrain_levels อยู่แล้ว → ทุก task มี curriculum พื้นด้วย

__init__.py

register 5 tasks ด้วย loop

G1-Curriculum-{Base,Reward,Push,Velocity,Full}

TRAIN · FINE-TUNE 300 ITERATIONS

เริ่มจาก G1 Rough ที่เดินเป็นแล้ว — curriculum ทำงานต่อจากตรงนั้น

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Full --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 agent.run_name=full300
# iteration ต่อจาก checkpoint: 5000 → 5299 · ~1.1 s/iteration บน RTX 3060 Laptop → ~6 นาที
Curriculum/terrain_levels: 2.43   Curriculum/feet_air_time_ramp: 0.25
Curriculum/push_strength:  0.00   Curriculum/command_vel_x:      1.00
$ uv run tensorboard --logdir logs/rsl_rl/g1_curriculum
```

ทำไม fine-tune

เทรนจากศูนย์ไม่ไหว

G1 Rough ของ Isaac Lab = 5,000 iterations × 4,096 ตัว · โน้ตบุ๊กทำได้ ~1,024 ตัว

ข้อควรระวัง

นาฬิกา curriculum เริ่ม 0 ใหม่

iteration ใน log เริ่ม 5000 แต่ common_step_counter เริ่ม 0 → ramp นับจากรอบนี้

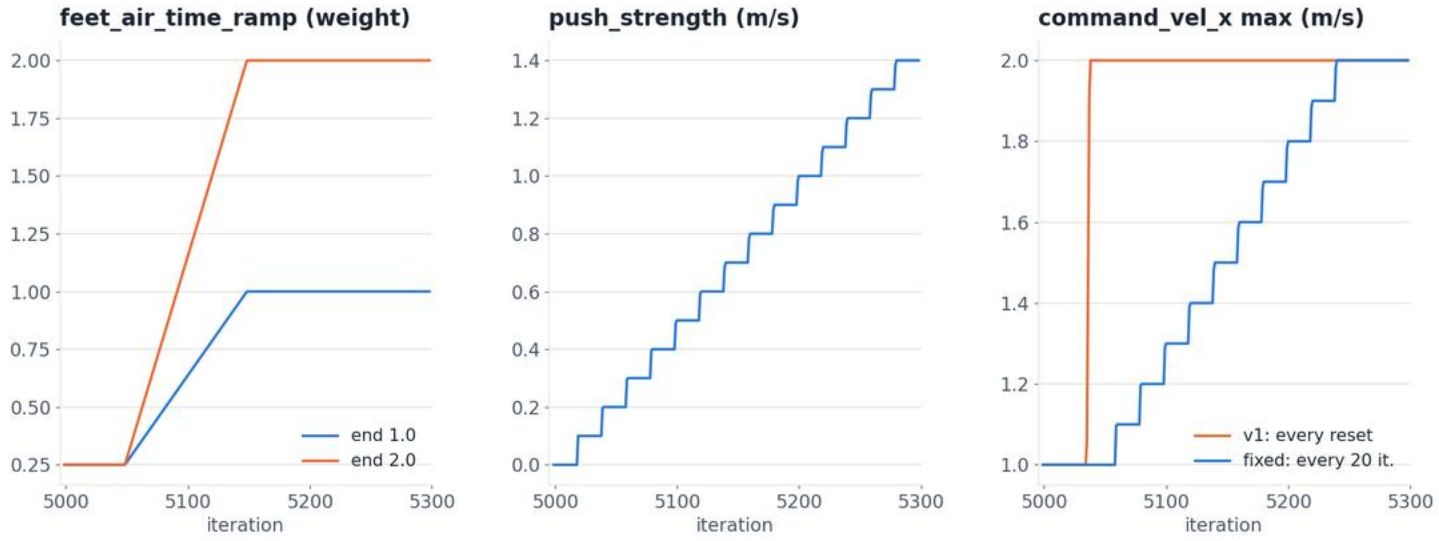
เทียบให้ยุติธรรม

มี Base เป็นตัวควบคุม

Base = fine-tune 300 it. เท่ากัน แต่ไม่มี term ใหม่

TENSORBOARD · CURRICULUM/*

curriculum ทำงานจริงไหม — ดูกราฟค่าที่ term คืนมา



`feet_air_time_ramp`

0.25 → 1.0

ขึ้นเป็นเส้นตรงช่วง it. 5050 → 5150 ตามที่ตั้งไว้

`push_strength`

0 → 1.4 m/s

ขึ้นทีละ 0.1 ทุก 20 it. ไม่เคยลดเลย (ล้ม < 5 % ตลอด) — ถึง 1.4 ตอนจบ

`command_vel_x`

1.0 → 2.0 m/s

ฉบับแก้: ขึ้นทีละ 0.1 ทุก 20 it. ถึง 2.0 ที่ it. 5240 · ฉบับแรกถึง 2.0 ใน 4 it. (หน้าถัดไป)

BUG ที่เจอ · COMMANDRANGE ฉบับแรก

ถูกเรียก “ทุกครั้งที่มีหุ่น reset” ไม่ใช่ทุก episode

ฉบับแรก · +0.1 ทุกครั้งที่ถูกเรียก

```
per_second = mean(episode_sums[env_ids]) / 20.0
if per_second > threshold * weight:
    hi = min(hi + step, final_max)   # ทุกครั้ง!
```

ฉบับแก้ · สะสม แล้วตัดสินทุก update_every step

```
self._sum += sums.sum() / 20.0; self._count += len(env_ids)
if counter - self._last_update ⩾ update_every:
    if self._sum / self._count > threshold * weight:
        hi = min(hi + step, final_max)
    self._sum, self._count, self._last_update = 0, 0, counter
```

v1

1.0 → 2.0 m/s ใน 4 it.

it. 5035–5038 · ตอน episode แรก ๆ เริ่มจบ หุ่นจบไม่พร้อมกัน → ถูกเรียกหลายสิบครั้งต่อ iteration

fix

ทีละ 0.1 ทุก 20 it.

ถึง 2.0 ที่ it. 5240 (กราฟขวาของหน้าก่อน)

ทำไมไม่เจอใน push

AdaptivePush มี update_every ตั้งแต่แรก

term ที่ตัดสินจาก “ทุก reset” ต้องมีตัวคุมความถี่เสมอ

ผล LAB 7.1 · 300 IT. × 1,024 ตัว · ~5.5 นาที/รัน

curriculum ขึ้นครบตามแผน — แต่พฤติกรรมเปลี่ยนน้อย

run	ค่าสุดท้ายของ curriculum	Mean reward	v_x จริง สั่ง 1.0	v_x จริง สั่ง 2.0	air/ก้าว (s) สั่ง 0.8	ล้ม · ผลัก 1.0 m/s	ล้ม · ผลัก 2.0 m/s
pretrained (ไม่ fine-tune)	–	–	0.845	1.867	0.110	1	9
Base (ไม่มี term ใหม่)	air 0.25 · push 0.5* · v_x≤1.0	23.2	0.834	1.846	0.119	1	8
Reward ramp	air 1.0 · push 0.5* · v_x≤1.0	23.8	0.833	1.829	0.122	1	–
Adaptive push	air 0.25 · push 1.4 · v_x≤1.0	22.5	0.802	1.845	0.120	2	16
Command range	air 0.25 · push 0.5* · v_x≤2.0	19.7	0.842	1.794	0.111	0	–
Full (3 terms)	air 1.0 · push 1.4 · v_x≤2.0	19.9	0.846	1.834	0.116	0	19

* push ±0.5 m/s ทุก 10–15 s คือค่าเดิมของ G1 Rough · eval_g1.py บน rough terrain (play mode) · 64 ตัว × 9 s ต่อคำสั่ง · ผลักทุก 2 s ทิศสุ่ม · รันละครั้งเดียว — ต่างกัน 1–2 ครั้งถือว่าเท่ากัน

อ่านผลให้ถูก

Mean reward ของแต่ละ run เทียบกันตรง ๆ ไม่ได้

โจทย์เปลี่ยน

reward เปลี่ยนตาม curriculum

Velocity / Full ได้ Mean reward 19.7–19.9 ต่ำกว่า Base 23.2 เพราะต้องตามคำสั่งถึง 2.0 m/s — โจทย์ยากขึ้น ไม่ใช่ policy แย่ลง

วัดด้วย `eval`

เทียบที่คำสั่งเดียวกัน

สั่ง 1.0 / 2.0 m/s ทุก run ได้ 0.80–0.85 / 1.77–1.87 m/s — 300 it. บน policy ที่เทรนมาแล้ว 5,000 it. เปลี่ยนพฤติกรรมได้น้อย

ราคาของ `curriculum`

ไม่ได้มีแต่ข้อดี

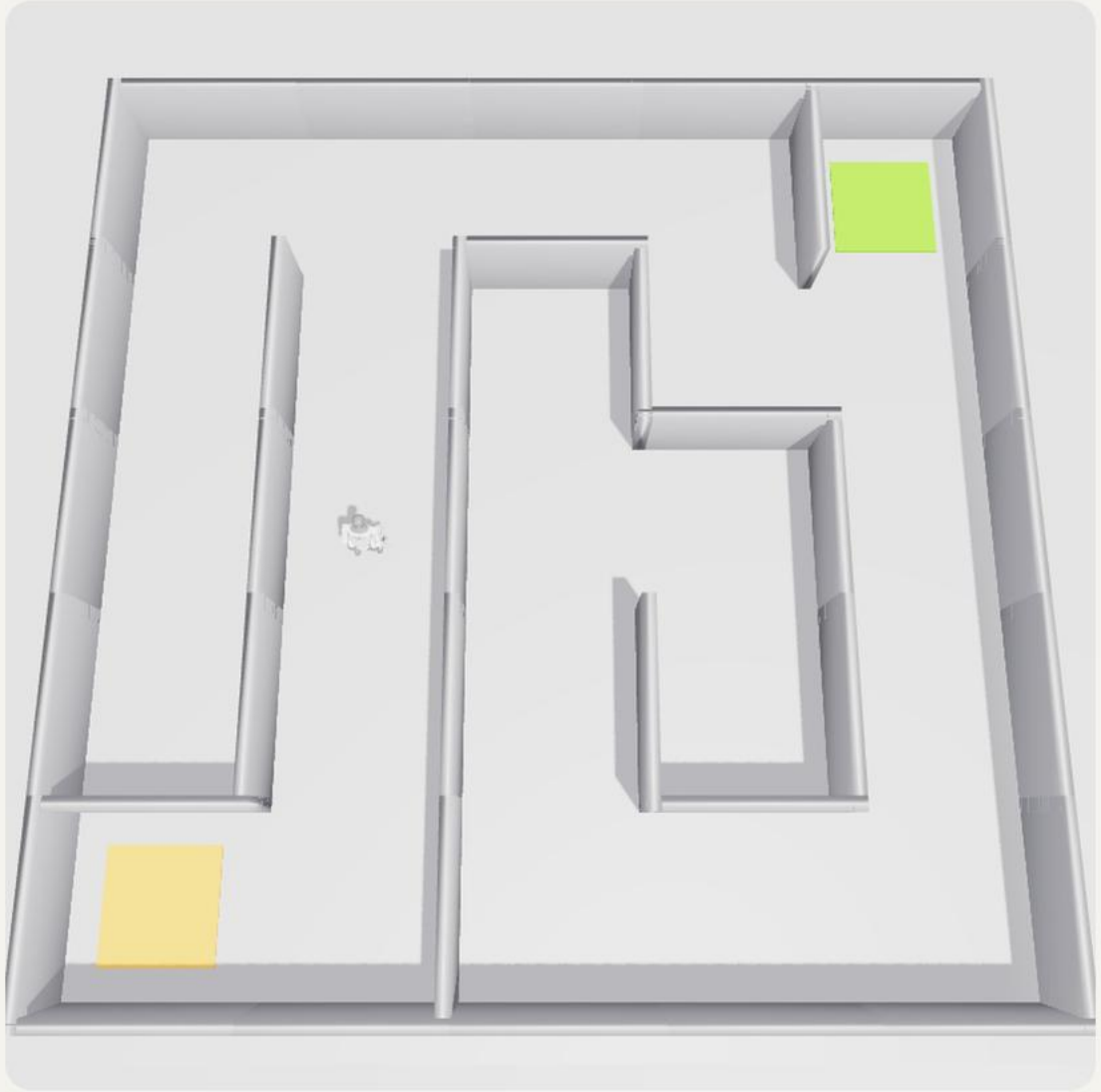
Push curriculum ล้ม 16 ครั้งเมื่อผลัก 2.0 m/s (Base 8) — เพราะเริ่มที่ 0 m/s ทั้งที่ G1 Rough เคยโดนผลัก 0.5 อยู่แล้ว → ครึ่งแรกของรอบ **ง่ายกว่าเดิม**

สรุป Lab 7.1: curriculum ทำงานตามโค้ด (กราฟขึ้นตามแผน) แต่ต้องวัดพฤติกรรมเสมอ · จุดเริ่มต้องไม่ง่ายกว่าที่ policy เคยเจอ · ถ้า curriculum ไม่เคย “ถอย” เลย แปลว่าเกณฑ์หลวมเกินไป

LAB 7.2 · CHALLENGE

G1 Maze Challenge

ไม่มีคนกดคีย์บอร์ดแล้ว — G1 ต้องหาทางจากมุมล่างซ้ายไปมุมบนขวาด้วย sensor ของตัวเอง



G1-Maze-Challenge ใน Newton viewer · กำแพงสูง 1 m · ช่องละ 2 m

CHALLENGE RULES

จาก S ไป G ด้วย sensor ของตัวเอง

แผนที่	5 × 5 ช่อง · 10 × 10 m · S = มุมล่างซ้าย หันไป +x · G = มุมบนขวา
สำเร็จ	ลำตัวเข้าใกล้กลางช่อง G ไม่เกิน 0.6 m ภายใน 150 s
ชน / ล้ม	torso · pelvis · ข้อศอก · ฝ่ามือ แตะอะไรก็ตาม → กลับไปเริ่มใหม่ (นับครั้ง)
sensor	maze_scanner (height scan) หรือ wall_lidar เท่านั้น — ห้ามอ่านแผนที่จริงจาก terrain
หุ่น	G1 + policy เดิน G1 Rough (fine-tune ด้วย curriculum ได้)
วัดผล	G1-Maze-Random · 16 ตัว · seed ที่ผู้สอนเลือกวันส่งงาน

SYSTEM

autopilot = 3 ชั้นบน policy ที่เดินเป็นอยู่แล้ว

maze_scanner
height scan 33 × 33 จุด

→

perception
กำแพง 4 ด้านของช่อง

→

planner
left / right / flood

→

controller
v_x, w_z แบบนุ่ม

→

G1 Rough policy
37 ข้อต่อ · 50 Hz

`autopilot.py`

เปลี่ยนแค่ "มือ"

เหมือน play_interactive.py ใน Week 3 — แต่คำสั่งความเร็วมาจากโค้ด ไม่ใช่คีย์บอร์ด

`2 height scanners`

คนละหน้าที่

ของ policy: 1.6 × 1.0 m ดูพื้นใต้เท้า · ของ autopilot: 3.2 × 3.2 m ดูกำแพงรอบตัว

`ช่องละ 2 m`

ตัดสินใจทีละช่อง

ถึงกลางช่อง (± 0.35 m) → อ่านกำแพง → เลือกช่องถัดไป

แนวคิด: policy เดินไม่ได้เรียนเรื่องเขาวงกตเลย — งานนี้คือ **hierarchical control**: ชั้นบนวางแผน ชั้นล่าง (RL) ทรงตัวและก้าวเดิน

TERRAINS/MAZE.PY

เขาวงกต = sub-terrain ใหม่ของ terrain generator

```
def add_wall(cx: float, cy: float, sx: float, sy: float):
    tf = trimesh.transformations.translation_matrix((cx, cy, 0.5 * h))
    meshes.append(trimesh.creation.box((sx, sy, h), tf))

for i in range(n):
    for j in range(n):
        # west and south walls of every cell + east / north walls on the outer
border
        if walls[i, j, 2]:
            add_wall(x0 + i * c, y0 + (j + 0.5) * c, t, c + t)
        if walls[i, j, 3]:
            add_wall(x0 + (i + 0.5) * c, y0 + j * c, c + t, t)
        if i == n - 1 and walls[i, j, 0]:
            add_wall(x0 + n * c, y0 + (j + 0.5) * c, t, c + t)
        if j == n - 1 and walls[i, j, 1]:
            add_wall(x0 + (i + 0.5) * c, y0 + n * c, c + t, t)

origin = np.array([x0 + 0.5 * c, y0 + 0.5 * c, 0.0])
return meshes, origin
```

`trimesh.creation.box`

กำแพง = กล่อง

หนา 0.15 m · สูง 1.0 m · ช่องละ 2 m (ทางเดินกว้าง 1.85 m)

`origin`

จุดเกิด = กลางช่อง (0, 0)

เป้า = ช่อง (4, 4) → ห่างจากจุดเกิด (8 m, 8 m) ทุกเขาวงกต

`difficulty`

ใช้ curriculum ได้

difficulty 0 → ทุบกำแพงทิ้ง 30 % · 1.0 → perfect maze (ทางเดียว)

TASKS/MAZE/ENV_CFG.PY

วาดเขาวงกตเป็นตัวอักษร — หรือสุ่มใหม่ทุกแผน

```
# The fixed challenge maze (5 x 5 cells of 2
m). Start S = bottom-left, goal G = top-
right.
# Shortest route: 10 cells. Left-hand wall
following: 18 cells. Right-hand: 30 cells.
CHALLENGE_LAYOUT = """
+-+-+-+-+-+
|       |G|
+ + +-+ + +
| | | |   |
+ + + +-+ +
| | |   | |
+ + + + + +
| | | | | |
+-+ + +-+ +
|S  |     |
+-+-+-+-+-+
"""
```

G1-Maze-Challenge

แผนที่ตายตัว 1 แผน

ทางสั้นสุด 10 ช่อง · เกาะซ้าย 18 ช่อง · เกาะขวา 30 ช่อง

G1-Maze-Random

สุ่ม 4 × 4 = 16 แผน

generate_maze() = randomized DFS → perfect maze แล้วทุบกำแพงออกตาม difficulty (แถวบน = ยากสุด)

G1-Maze-Open

ไม่มีกำแพง

พื้น mesh เปล่า ๆ — ใช้ทดสอบ policy เดิน (Exercise 7.3)

SENSOR · RAYCASTERCFG

height scanner 3.2 × 3.2 m รอบตัว

```
cfg.scene.maze_scanner = RayCasterCfg(
    prim_path="{ENV_REGEX_NS}/Robot/torso_link",
    offset=RayCasterCfg.OffsetCfg(pos=(0.0, 0.0, 20.0)),
    ray_alignment="yaw",
    pattern_cfg=patterns.GridPatternCfg(resolution=0.1,
size=[3.2, 3.2]),
    mesh_prim_paths=["/World/ground"],
    global_world_only=True,
    update_period=cfg.decimation * cfg.sim.dt,
    debug_vis=False,
)
```

`offset z = 20 m`

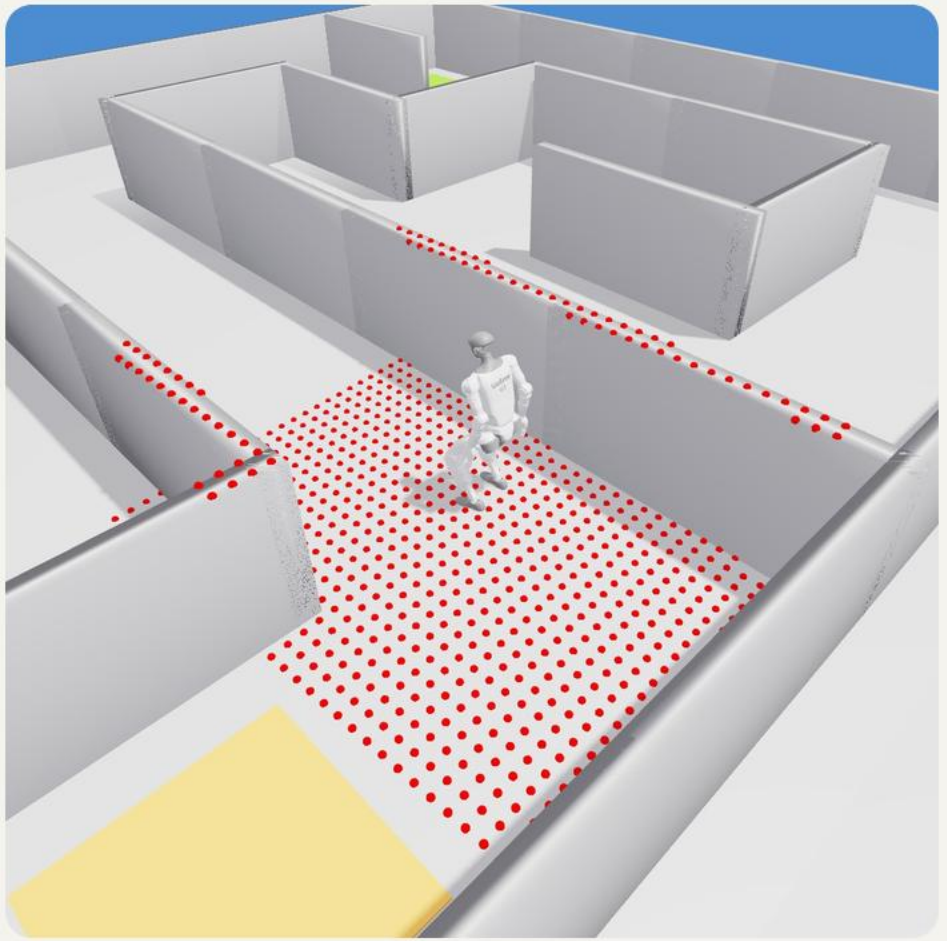
ยิงจากฟ้าลงพื้น

hit z ≈ 0 = พื้น · ≈ 1.0 = บนกำแพง

`ray_alignment=yaw`

หมุนตาม yaw อย่างเดียว

หุ่นเอียงตัวเดิน grid ก็ไม่เอียงตาม



MAZE_SHOW_SCAN=1 · จุดที่ rays ชน (debug vis ของ ray caster)

LAYER 1 · PERCEPTION

หากำแพง: นับจุดสูง > 0.5 m ใน “กล่องบาง” แต่ละด้านของช่อง

```
centre = self.centre_of(cell).unsqueeze(1)  # (N, 1, 2)
rel = hits_xy - centre  # (N, R, 2)
tall = hits_z > ground_z + 0.5
out = torch.zeros(hits_xy.shape[0], 4, dtype=torch.bool,
device=self.device)
half = 0.5 * self.c
for d, (dx, dy) in enumerate(DIRS):
    along = rel[..., 0] * dx + rel[..., 1] * dy  # distance towards
side d
    across = rel[..., 0] * dy - rel[..., 1] * dx
    in_box = (torch.abs(along - half) < 0.15) & (torch.abs(across) <
0.6)
    out[:, d] = (in_box & tall).sum(dim=1) ⩾ 3
return out
```

ray_hits_w

จุดที่ rays ชน (x, y, z)

หักด้วย env_origins → พิกัดเทียบจุดเกิด · ไม่สน yaw ของหุ่น

กล่องบาง · ≥ 3 จุด

ลึก 0.3 m · ยาว 1.2 m

ตรงขอบช่องด้านละกล่อง · 3 จุดขึ้นไปกันจุดสูงหลงจากมุมกำแพงข้างเคียง

LAYER 2 · PLANNER

เลือกช่องถัดไป: เกาะกำแพง หรือ flood fill

```
if self.strategy in ("left", "right"):
    # wall following: try left, straight, right, back (mirror for the right hand)
    turns = (1, 0, 3, 2) if self.strategy == "left" else (3, 0, 1, 2)
    choice = (h + 2) % 4
    for t in reversed(turns):  # the first open side in the preference list wins
        d = (h + t) % 4
        choice = torch.where(open_sides.gather(1, d.unsqueeze(1)).squeeze(1), d,
choice)
    return choice
```

```
dist = self.flood_fill(walls)
idx = torch.arange(len(h), device=self.device)
best = torch.full_like(h, -1)
best_cost = torch.full(h.shape, 10**6, dtype=torch.long, device=self.device)
# straight first, so ties keep the robot walking straight (fewer turns)
for t in (0, 1, 3, 2):
    d = (h + t) % 4
    nb = (cell + self._dirs[d]).clamp(0, self.n - 1)
    cost = dist[idx, nb[:, 0], nb[:, 1]]
    ok = open_sides[idx, d] & (cost < best_cost)
    best = torch.where(ok, d, best)
    best_cost = torch.where(ok, cost, best_cost)
```

`left / right`

มือซ้ายแตะกำแพง

ลำดับ: ซ้าย → ตรง → ขวา → กลับหลัง · ไม่ต้องจำแผนที่ · รับประกันว่าถึงเป้าที่อยู่บนขอบ

`flood`

flood fill (Micromouse)

คิดระยะทุกช่องถึงเป้า โดยถือว่ากำแพงที่ **ยังไม่เห็น** = ทางเปิด · เดินไปช่องที่ระยะน้อยสุด · เจอกำแพงใหม่ก็คิดใหม่

`self.walls`

แผนที่ที่หุ่นจำได้

−1 ยังไม่รู้ · 0 เปิด · 1 กำแพง (ขอบนอกรู้ตั้งแต่ต้น)

LAYER 3 · CONTROLLER

เดินไปกลางช่องถัดไป — ชะลอตอนเลี้ยว ไม่หมุนอยู่กับที่

```
to_target = self.centre_of(self.target) - pos_xy
yaw_err = torch.atan2(to_target[:, 1], to_target[:, 0]) - yaw
yaw_err = torch.atan2(torch.sin(yaw_err), torch.cos(yaw_err))
# full speed when facing the target, down to 0.15 m/s when it is 90 deg or more
to the side
v_des = 0.15 + (self.speed - 0.15) * torch.clamp(torch.cos(yaw_err), min=0.0) **
2
w_des = torch.clamp(1.5 * yaw_err, -self.max_turn, self.max_turn)
# rate limits [m/s^2], [rad/s^2]
self.cmd[:, 0] += torch.clamp(v_des - self.cmd[:, 0], -self.max_acc * dt,
self.max_acc * dt)
self.cmd[:, 2] += torch.clamp(w_des - self.cmd[:, 2], -self.max_ang_acc * dt,
self.max_ang_acc * dt)
return self.cmd.clone()
```

`cos²(yaw error)`

หันตรง = เต็มสปีด

หันผิด 90° ขึ้นไป → เหลือ 0.15 m/s แต่ยังก้าว (ไม่หมุนอยู่กับที่) · w_z ≤ 0.8 rad/s ในช่วงที่ G1 Rough เทรนมา

`rate limit`

0.8 m/s² · 2 rad/s²

ปิดได้ด้วย MAZE_ACC=1e9,1e9 — วัดแล้วไม่ล้มเหมือนกัน

ผลวัด: ตอนแรกคิดว่าคำสั่งกระตุกทำให้ล้ม จึงใส่ rate limit — แต่ปิดแล้วก็ **ล้ม 0 ครั้ง** (flood 39.4 s vs 41.3 s · left 73.3 vs 75.7 s · 16 ตัว) ตัวการจริงคือ policy Flat บนพื้น mesh (หน้า “สิ่งที่เจอระหว่างทำ”) — ตั้งสมมติฐานแล้วต้องวัดเสมอ

RUN

ดูหุ่นเดินเอง แล้ววัดผลแบบ headless

```
$ uv run python scripts/maze_autopilot.py --task G1-Maze-Challenge --num_envs 1 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt --viz newton
# MAZE_STRATEGY=left|right|flood  MAZE_SENSOR=height|lidar  MAZE_SPEED=0.6  MAZE_CAM=chase|top
[MAZE] t= 10.0s robot0 cell=[1, 1] cells walked=2
[MAZE] robot 0: GOAL at 41.2 s after 10 cells
[MAZE] map seen by robot 0 ('?' = never sensed):
```

```
+-+-+-+-+-+
|       |G|
+?+ +-+ + +
| | | |   |
+?+ +?+-+ +
| | | ? ? |
+?+ +?+?+?+
| | | ? ? |
+-+ +?+?+?+
|S  | ? ? |
+-+-+-+-+-+
```

flood · 10 ช่อง

เห็นกำแพงแค่ที่ผ่าน

'?' = ด้านที่หุ่นไม่เคยยืนดู — flood fill ถือว่าเปิด แล้วโชคดีที่ทางนั้นถูก

--num_envs 16 · ไม่ใส่ --viz

เร็วกว่าเวลาจริง ~2×

150 s ในซิม ≈ 80 s บนโน้ตบุ๊ก · ได้สถิติ 16 ตัวในครั้งเดียว

ผล LAB 7.2 · PRETRAINED G1 ROUGH · 0.6 M/S · 16 ตัว

ทุก strategy ถึงเป้า 16/16 โดยไม่ล้มเลย

strategy · sensor	ถึงเป้า	เวลาเฉลี่ย	ช่วง	ช่องที่เดิน	ชน/ล้ม
left · height	16/16	75.7 s	74.3 – 76.9	18	0
right · height	16/16	130.9 s	128.6 – 134.2	30	0
flood · height	16/16	41.3 s	40.6 – 42.6	10	0
left · lidar	16/16	75.4 s	73.9 – 76.8	18	0
left · 16 random	16/16	77.9 s	30.3 – 104.7	17.9	0
flood · 16 random	16/16	49.0 s	30.0 – 73.9	11.8	0

flood

เร็วสุด

41 s · เดินตามทางสั้นสุด 10 ช่องพอดีในแผนที่ challenge

left vs right

ขึ้นกับแผนที่

แผนที่นี้ซ้ายได้เปรียบ (18 vs 30 ช่อง) · บางแผนที่กลับกัน

16 random

flood ยังชนะ

เฉลี่ย 11.8 ช่อง vs 17.9 ช่อง (left)

headless · ช่องละ 2 m · เวลารวมการเลี้ยว · พอดีเป้า ≤ 0.6 m

ลองเร่งเป็น 1.0 M/S · FLOOD · 16 RANDOM

เร็วขึ้น ≠ คะแนนดีขึ้น

policy เดิน	MAZE_SPEED	ถึงเป้า	เวลาเฉลี่ย	ชน/ล้ม	score*
pretrained G1 Rough	0.6	16/16	49.0 s	0	51.0
pretrained G1 Rough	1.0	15/16	43.2 s	21	−54.5
Velocity curriculum (fix)	1.0	15/16	45.0 s	18	−41.3
Full curriculum (fix)	1.0	14/16	41.6 s	21	−59.1

1.0 m/s

เร็วขึ้น ~6 s แต่ล้ม 18–21 ครั้ง

ทางตรงไม่น่าใช่สาเหตุ: eval สั่งเดินตรง 2.0 m/s ล้ม 0 ทุก policy → สงสัยช่วงเลี้ยวมุม (ยังไม่ได้พิสูจน์ — MAZE_DEBUG=1 + viewer)

curriculum 300 it.

ไม่ช่วยในโจทย์นี้

policy ที่ fine-tune ด้วย Velocity / Full ยังล้มพอ ๆ กับ pretrained ที่ 1.0 m/s

โจทย์ challenge

ตรงนี้คือช่องให้ทำคะแนน

ชะลอก่อนเลี้ยว · เร่งในทางตรง · เทรน curriculum เรื่องการเลี้ยว (w_z)

* score = 100 × ถึงเป้า/16 − เวลาเฉลี่ย − 5 × ชน/ล้ม (หน้า challenge)

สิ่งที่เจอระหว่างทำ

policy Flat ของ Week 3 ล้มบนพื้นเขาวงกต — Rough ไม่ล้ม

policy	พื้น	0.4 m/s + 0.8 rad/s	0.6 m/s + 1.0 rad/s	0.15 m/s + 0.8 rad/s
G1 Flat (Week 3)	plane (Isaac-Velocity-Flat-G1)	0	0	0
G1 Flat (Week 3)	mesh (G1-Maze-Open-FlatPolicy)	19	17	0
G1 Rough	mesh (G1-Maze-Open)	0	0	0

ล้มต่อ 64 ตัว · 15 s

พื้นเรียบเหมือนกัน แต่คนละชนิด

plane = ระนาบคณิตศาสตร์ · terrain generator = กล่อง mesh — contact ต่างกัน

Flat ทั้งเขาวงกต

15/16 ถึงเป้า · ล้ม 24 ครั้ง

flood · 16 ตัว · ตัวเดียวกันกับ Rough ที่ได้ 16/16 ล้ม 0

Rough

ผ่าน terrain curriculum

เทรนบน mesh หลายระดับความยาก → ทนกว่า · เลยใช้เป็น policy เดินของ challenge

บทเรียน: domain gap เล็ก ๆ (plane → mesh) ก็ทำให้ policy พังได้ — curriculum ที่พาไปเจอสภาพหลากหลายระหว่างเทรนคือวิธีกันปัญหานี้

SENSOR ที่สอง · WALL_LIDAR

lidar 2D: 72 rays แนวนอน ทุก 5° ที่ระดับอก

```
# ray_alignment="yaw" rotates only the ray start points (it was
made for height scanners),
# so we take each ray's world angle from its hit point; rays that
hit nothing are ignored
rel = lidar.data.ray_hits_w.torch[..., :2] -
lidar.data.pos_w.torch[:, None, :2]
dist = torch.nan_to_num(torch.linalg.norm(rel, dim=-1), nan=1e6,
posinf=1e6)
angles_w = torch.atan2(rel[..., 1], rel[..., 0])
walls_now = pilot.walls_from_lidar(dist, angles_w, pos_xy -
pilot.centre_of(cell))
```

bug ที่เจอ

ทิศ rays ไม่หมุนตามหุ่น

ray_alignment="yaw" หมุนแค่ **จุดเริ่ม** ของ ray (ทำมาให้ height scanner) · ตอนแรกบวก yaw ซ้ำ → กำแพงเพี้ยน 90° → ชน 208 ครั้ง 0/16

แก้

คิดมุมจากจุดที่ชน

atan2 ของ (hit – ตำแหน่ง sensor) → 16/16 ถึงเป้า · ชน 0

CHALLENGE · ส่งงาน

ทำให้ G1 ออกจากเขาวงกตเร็วที่สุด

แก้ได้

autopilot.py ทั้ง 3 ชั้น

perception (sensor ไหน, กี่จุด) · planner (strategy ใหม่) · controller (สปีด, การเลี้ยว)

แก้ได้

policy เดิน

fine-tune G1 Rough ด้วย curriculum ของ Lab 7.1 ให้เดินเร็ว/ทนขึ้น

ห้าม

อ่านแผนที่จริง

ต้องรู้กำแพงจาก sensor เท่านั้น · ห้ามแก้ termination / ขนาดเขาวงกต

```
$ MAZE_STRATEGY=flood uv run python scripts/maze_autopilot.py --task G1-Maze-Random --num_envs 16 \
    --checkpoint <policy ของคุณ> env.scene.terrain.terrain_generator.seed=<seed วันส่งงาน>
[MAZE] RESULT strategy=flood sensor=height: 16/16 reached the goal, time mean 49.0 s ... crashes/falls 0
```

Score: คะแนน = 100 × (จำนวนที่ถึงเป้า / 16) − เวลาเฉลี่ย (s) − 5 × จำนวนครั้งที่ชน/ล้ม · baseline (flood, 0.6 m/s) = 100 − 49.0 − 0 = **51.0**

EXERCISES · WEEK 7

ลงมือทำ

4 โจทย์ · ทุกข้อรันได้บนโน้ตบุ๊ก · เฉลย (พื้นเขียว) ต่อท้ายแต่ละข้อ

EXERCISE 7.1 · TUNE THE RAMP

ramp feet_air_time ให้สูงขึ้น เดินต่างไปไหม

7.1

```
$ uv run isaaclab train --rl_library rsl_rl --task G1-Curriculum-Reward --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=300 \
    agent.run_name=reward_w2 env.curriculum.feet_air_time_ramp.params.end_weight=2.0
```

โจทย์

1. ramp นี้เริ่มที่ iteration ไหนใน log และถึงค่าสุดท้ายที่ iteration ไหน? (log เริ่ม 5000 · start_step = 50 × 24)
2. วัด air time ต่อก้าว (eval_g1.py, สั่ง 0.8 m/s) ของ Base · Reward (1.0) · Reward (2.0) — เปลี่ยนไปทางไหน?
3. ทำไม ramp ถึงดีกว่าเปลี่ยน weight ทีเดียว (modify_reward_weight) — ดูกราฟ Mean reward ตอนเปลี่ยน

เฉลย · EXERCISE 7.1

end_weight 2.0 → ก้าวนานขึ้น ~10 % แต่เดินช้าลงเล็กน้อย

1. start_step = 50 × 24 = 1,200 step → เริ่มขึ้นที่ iteration **5050** และถึงค่าสุดท้ายที่ **5150** (150 × 24) · log เริ่ม 5000 แต่ common_step_counter เริ่ม 0
2. air time ต่อก้าวที่ 0.8 m/s: Base 0.119 s · end 1.0 → 0.122 s · end 2.0 → **0.132 s** · Episode_Reward/feet_air_time 0.032 → 0.075 — แต่ v_x ที่สั่ง 2.0 ลดจาก 1.846 → 1.770 m/s
3. modify_reward_weight เปลี่ยนทีเดียว → reward scale กระโดด critic ประเมินค่าผิดชั่วคราว · ramp ค่อย ๆ เปลี่ยนให้ critic ตามทัน (สไลด์นี้ไม่ได้รันแบบกระโดดเทียบ — ลองเป็นการบ้านเสริม)

run	weight	air (s)	v_x @2.0
Base	0.25	0.119	1.846
Reward · end 1.0	1.0	0.122	1.829
Reward · end 2.0	2.0	0.132	1.770

eval_g1.py · 64 ตัว · rough terrain

สรุป: reward weight ใหญ่ขึ้น = แลกความเร็วกับท่าก้าวยาว · ramp แค่ช่วยให้เปลี่ยนอย่างนุ่มนวล ไม่ได้เปลี่ยนว่าสุดท้ายแลกอะไร

EXERCISE 7.2 · YOUR OWN TERM

เขียน curriculum ด้วย modify_term_cfg (ไม่ต้องสร้าง class)

7.2

```
CurrTerm(func=mdp.modify_term_cfg,
    params={"address": "rewards.action_rate_l2.weight",
            "modify_fn": ???, "modify_params": {...}})
```

โจทย์

1. เขียน modify_fn(env, env_ids, old_value, **params) ให้ action_rate_l2 weight ไล่จาก −0.005 → −0.02 ใน 80 iterations
2. เพิ่มเป็น task ใหม่ G1-Curriculum-ActionRate แล้ว fine-tune 100 it. — กราฟ Curriculum/... ขึ้นตามที่ตั้งไหม?
3. ถ้า modify_fn คืน mdp.modify_term_cfg.NO_CHANGE จะเกิดอะไร และควรใช้เมื่อไร?

เฉลย · EXERCISE 7.2

modify_term_cfg ทำงาน — แต่ไม่ขึ้นกราฟ Curriculum

1. ดูโค้ดด้านขวา (อยู่ใน tasks/curriculum/env_cfg.py) · ทดสอบแล้ว: G1-Curriculum-ActionRate 100 it. ≈ 2 นาที

2. modify_term_cfg คืนค่า **None** → rsl_rl ไม่ log Curriculum/action_rate_weight เลย · เช็คผ่าน Episode_Reward/action_rate_l2: −0.49 (it. 5025) → −1.07 → −1.55 → **−1.85** (5098) เทียบ Base ≈ −0.51

3. NO_CHANGE = ไม่เรียก setter เลย (ไม่เขียนค่าซ้ำ) · ใช้เมื่อยังไม่ถึงเงื่อนไข เช่น ก่อน num_steps — คืน None ไม่ได้เพราะ None อาจเป็นค่าที่ตั้งใจเขียน

```
def action_rate_schedule(env, env_ids, old_value, start_step: int,
end_step: int, start: float, end: float):
    frac = min(max((env.common_step_counter - start_step) /
max(end_step - start_step, 1), 0.0), 1.0)
    return start + frac * (end - start)

self.curriculum.action_rate_weight = CurrTerm(
    func=mdp.modify_term_cfg,
    params={
        "address": "rewards.action_rate_l2.weight",
        "modify_fn": action_rate_schedule,
        "modify_params": {
            "start_step": 0,
            "end_step": 80 * STEPS_PER_IT,
            "start": -0.005,
            "end": -0.02,
        },
    },
)
```

สรุป: อยากเห็นกราฟ → เขียนเป็น class (ManagerTermBase) แล้ว return ค่า แบบ 3 terms ใน Lab 7.1

EXERCISE 7.3 · DOMAIN GAP

policy เดียวกัน พื้นเรียบเหมือนกัน — ทำไมล้ม?

7.3

```
$ EVAL_CMDS="0.4,0,0.8;0.6,0,1.0" uv run python scripts/eval_g1.py --num_envs 64 \
    --task Isaac-Velocity-Flat-G1 | G1-Maze-Open-FlatPolicy --checkpoint pretrained/g1_flat/model_pretrained.pt
$ ... --task G1-Maze-Open --checkpoint pretrained/g1_rough/model_pretrained.pt
```

โจทย์

1. รันทั้ง 3 แบบ (สั่งเดินเป็นวงกลม ไม่ชนขอบ) นับจำนวนล้มต่อ 64 ตัว · 15 s
2. พื้น 2 แบบต่างกันตรงไหน (ดู terrain_type ใน G1FlatEnvCfg กับ maze_terrain_cfg)? ทำไม policy Rough ถึงไม่ล้ม?
3. ถ้าต้องใช้ policy Flat ในเขาวงกตจริง ๆ จะแก้ที่ environment หรือที่การเทรน? เสนอ 1 วิธีที่ใช้ curriculum

เฉลย · EXERCISE 7.3

plane กับ mesh คือคนละพื้นสำหรับ policy

1. ล้มต่อ 64 ตัว · 15 s (0.4+0.8 / 0.6+1.0 / 0.15+0.8): Flat บน plane **0 / 0 / 0** · Flat บน mesh **19 / 17 / 0** · Rough บน mesh **0 / 0 / 0**

2. G1FlatEnvCfg ใช้ terrain_type="plane" (ระนาบไม่มีที่สิ้นสุด contact แบบ analytic) · เขาวงกตใช้ terrain generator = กล่อง mesh — contact ต่างกันเล็กน้อยแต่พอทำให้ policy ที่ไม่เคยเห็นล้ม · Rough เทรนบน mesh หลายระดับผ่าน terrain_levels

3. แก้ที่การเทรน: fine-tune Flat บน terrain generator โดยเริ่มจากแถวเรียบ (difficulty 0) แล้วให้ terrain_levels พาไปพื้นยาก = curriculum · หรือสุ่มแรงเสียดทาน/พื้นระหว่างเทรน

สรุป: domain gap ไม่จำเป็นต้องใหญ่ถึงจะทำให้ policy พัง — เทรนบนสภาพที่หลากหลายกว่าที่ใช้จริงไว้ก่อน

EXERCISE 7.4 · SENSOR DESIGN

scanner หยาบลง / เล็กลง — autopilot ยังเห็นกำแพงไหม

7.4

```
$ MAZE_STRATEGY=flood uv run python scripts/maze_autopilot.py --task G1-Maze-Challenge --num_envs 16 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt env.scene.maze_scanner.pattern_cfg.resolution=0.2
# ลองต่อ: resolution=0.15 · pattern_cfg.size=[2.0,2.0] · MAZE_SENSOR=lidar
```

โจทย์

1. กำแพงหนา 0.15 m — ที่ resolution 0.1 / 0.15 / 0.2 m จะมี ray ตกบนสันกำแพงกี่แถว? คาดผลก่อนรัน
2. รันทั้ง 3 ค่า + size 2.0 m บันทึก ถึงเป้า / เวลา / ชน แล้วอธิบายตัวที่พัง
3. แก้ walls_from_height_scan ให้ทนกับ scan หยาบขึ้น (เช่น ขยายกล่อง หรือลดจำนวนจุดขั้นต่ำ) แล้วรันใหม่

เฉลย · EXERCISE 7.4

resolution หยาบลงยังรอด — scanner เล็กลงพังทันที

1. rays ยิงดิ่ง จึงชนกำแพงได้แค่ที่สันบน (หนา 0.15 m): 0.1 → 1–2 แถว · 0.15 → 1 แถว · 0.2 → 0–1 แถว (อาจพลาดได้) · แต่แถวเดียวยาว 1.2 m มี 6–12 จุด ≥ 3 จุดผ่านเกณฑ์

2. flood · challenge · 16 ตัว: 0.1 → 16/16 · 41.3 s · 0 ชน · 0.15 → 16/16 · 41.3 s · 0 · 0.2 → 16/16 · 41.4 s · 0 · **size 2.0 m → 0/16 · ชน 439 ครั้ง** — grid ±1.0 m แต่กล่องตรวจกำแพงอยู่ที่ 0.85–1.15 m จากกลางช่อง หุ่นยืนเยื้องนิดเดียวก็มองไม่เห็นกำแพงฝั่งไกล

3. ตัวอย่างแก้: ขยับกล่องเข้ามา (0.85–1.0 m) หรือลด min hits เป็น 1 สำหรับ scan เล็ก · หรือใช้ lidar (ระยะ 6 m) — ลองเองแล้วรายงานผล

สรุป: ออกแบบ sensor จาก “สิ่งที่ต้องเห็น”: ขนาดต้องครอบคลุมกำแพงทั้ง 4 ด้าน + ระยะเยื้อง · ความละเอียดต้องเล็กกว่าความหนาของวัตถุ

ASSIGNMENT 7

ส่ง 2 ส่วน

7.A · Curriculum 1 ฟังก์ชัน

เขียน curriculum term ใหม่ 1 ตัว (function หรือ class) · อธิบายว่าปรับอะไร เงื่อนไขเพิ่ม/ลดความยากคืออะไร (เช่น “ถึง iteration 1,500 เพิ่ม feet_air_time”) · แนบกราฟ Curriculum/<ชื่อ> จาก TensorBoard

7.B · G1 Maze Challenge

autopilot.py ที่แก้แล้ว + checkpoint (ถ้า fine-tune) + บรรทัด [MAZE] RESULT บน G1-Maze-Random 16 ตัว · อธิบาย 1 หน้า: แก้อะไร ทำไมเร็วขึ้น/ชนน้อยลง

ไฟล์: โปรเจกต์ + checkpoint ทั้งหมดอยู่ใน Week07_Curriculum_Maze/g1_maze_lab · เฉลยโค้ดอยู่ในสไลด์ (พื้นเขียว)